Operating System: Process

Dr. Jit Muherjee

Concept of Process

- An operating system executes a variety of programs
  - batch systems - jobs
  - time-shared systems - user programs or tasks
  - Job, task and program used interchangeably
- A process is basically a program in execution.
  - The execution of a process must progress in a sequential fashion.
- A computer program is a collection of instructions that performs a specific task when executed by a computer.
  - a process is a dynamic instance of a computer program
- A process is defined as an entity which represents the basic unit of work to be implemented in the system.
- When a program is loaded into the memory and it becomes a process,
- The process is not as same as program code but a lot more than it. A process is an 'active' entity as opposed to the program which is considered to be a 'passive' entity.

Process and Program

More to a process than just a program:

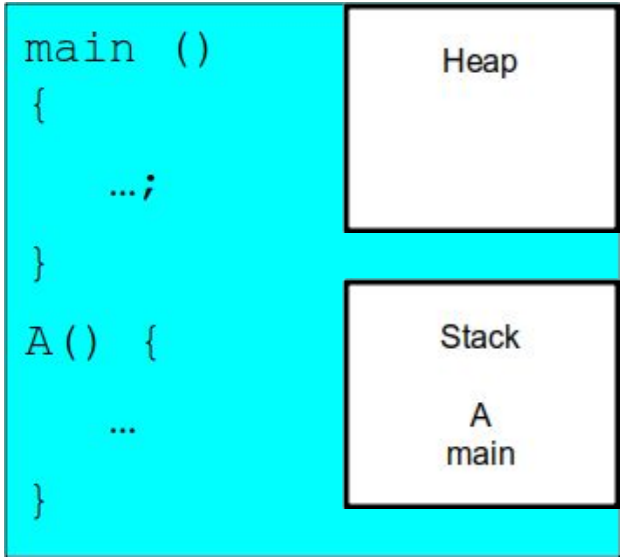
- Program is just part of the process state
- I can run Vim or Notepad on lectures.txt, you can run it on homework.java – Same program, different processes

Less to a process than

a program:

- A program can invoke more than one process
- A web browser launches multiple processes, e.g., one per tab

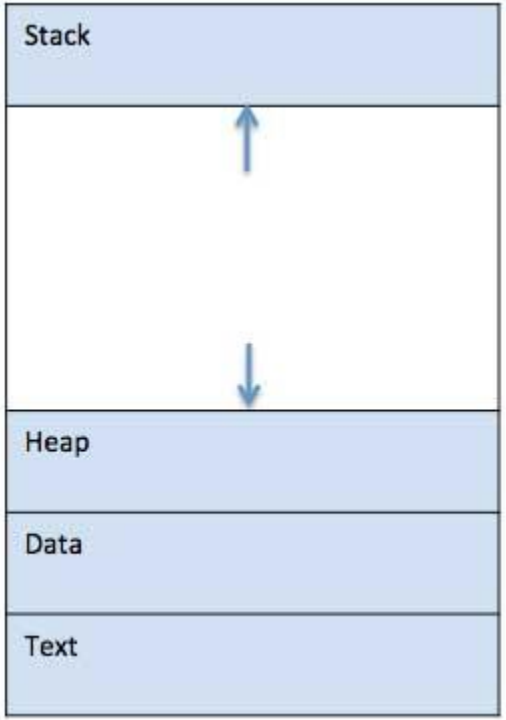
```
main ()
{
    …;
}
A() {
    …
}
```



Process Components

Process memory is divided into four sections for efficient working

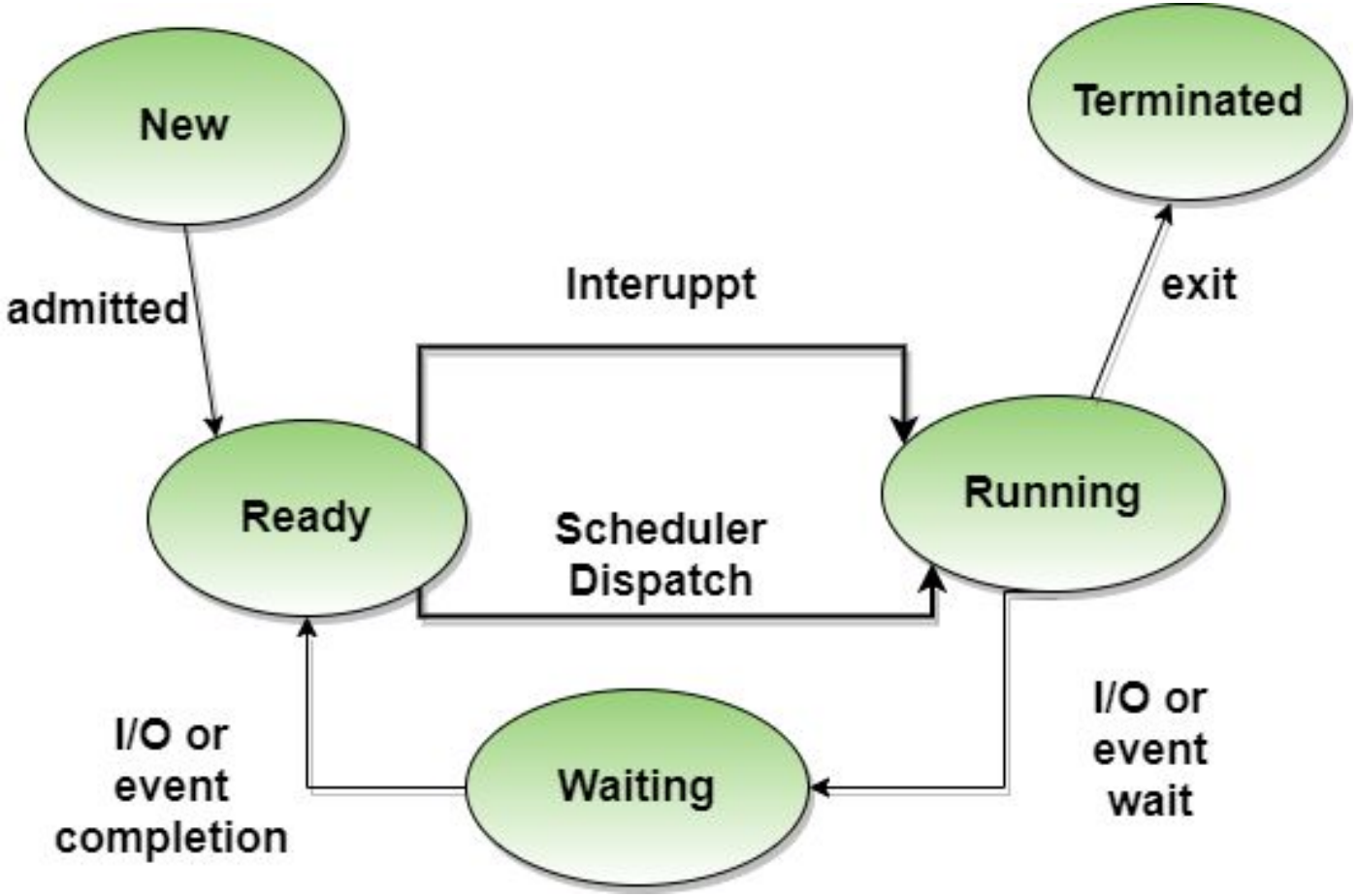
- Stack - The process Stack contains the temporary data such as method/function parameters, return address and local variables.
- Heap - This is dynamically allocated memory to a process during it run time.
    - managed via calls to new, delete, malloc, free, etc.
- Text - This includes the current activity represented by the value of Program Counter, the compiled program code and the contents of the processor's registers.
    - read in from non-volatile storage when the program is launched
- Data - This section contains the global and static variables.
    - allocated and initialized prior to executing the main.



Process Creation

- Processes are created and deleted dynamically
- Process which creates another process is called a parent process; the created process is called a child process.
- Result is a tree of processes
  - e.g. UNIX - processes have dependencies and form a hierarchy.
- Resources required when creating process -
  - CPU time, files, memory, I/O devices etc.
- Resource sharing
  - Parent and children share all resources.
  - Children share subset of parent's resources - prevents many processes from overloading the system.
  - Parent and children share no resources.
- Execution
  - Parent and child execute concurrently.
  - Parent waits until child has terminated.
- Address Space
  - Child process is duplicate of parent process.
  - Child process has a program loaded

Process State



Process Termination

- Process executes last statement and asks the operating system to delete it (exit).
  - Output data from child to parent (via wait).
  - Process' resources are deallocated by operating system.
- Parent may terminate execution of child processes.
  - Child has exceeded allocated resources.
  - Task assigned to child is no longer required.
  - Parent is exiting
    - OS does not allow child to continue if parent terminates
    - Cascading termination

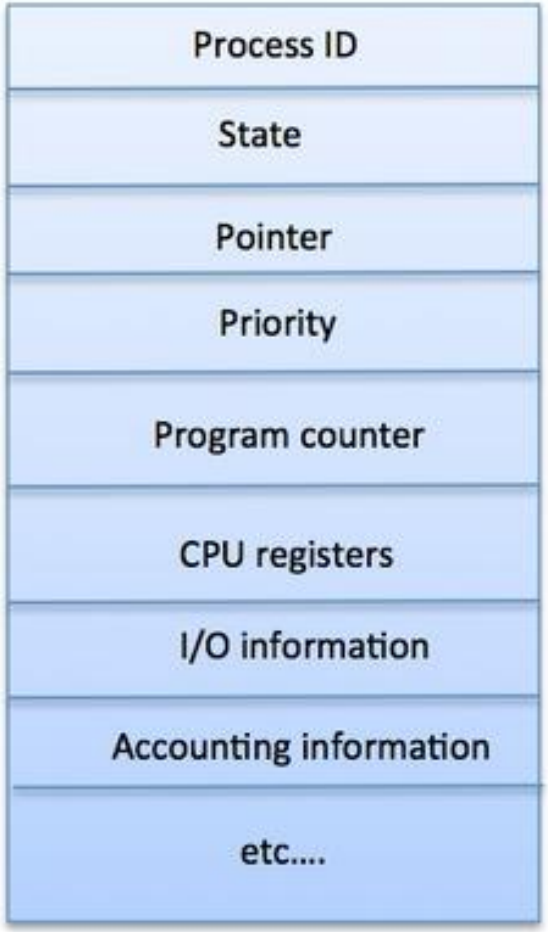
Process State

1. New or Start - This is the initial state when a process is first started/created.
2. Ready - The process is waiting to be assigned to a processor.
    a. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run.
    b. Process may come into this state after Start state or while running it by but interrupted by the scheduler to assign CPU to some other process.
3. Running - Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.
4. Waiting - Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.
5. Terminated or Exit - Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.

Process Control Block (PCB)

Process Control Block / task control block for each process, enclosing all the information about the process It is a data structure

1. **Process State** - The current state of the process i.e., whether it is ready, running, waiting, or whatever.
2. **Process privileges -** This is required to allow/disallow access to system resources.
3. **Process ID** - Unique identification for each of the process in the operating system.
4. **Pointer** - A pointer to parent process.
5. **Program Counter** - Program Counter is a pointer to the address of the next instruction to be executed for this process.
6. **CPU registers** - Various CPU registers where process need to be stored for execution for running state.
7. **CPU Scheduling Information** - Process priority and other scheduling information which is required to schedule the process.
8. **Memory management information** - This includes the information of page table, memory limits, Segment table depending on memory used by the operating system.
9. **Accounting information** - This includes the amount of CPU used for process execution, time limits, execution ID etc.
10. **Status information**- This includes a list of I/O devices allocated to the process.

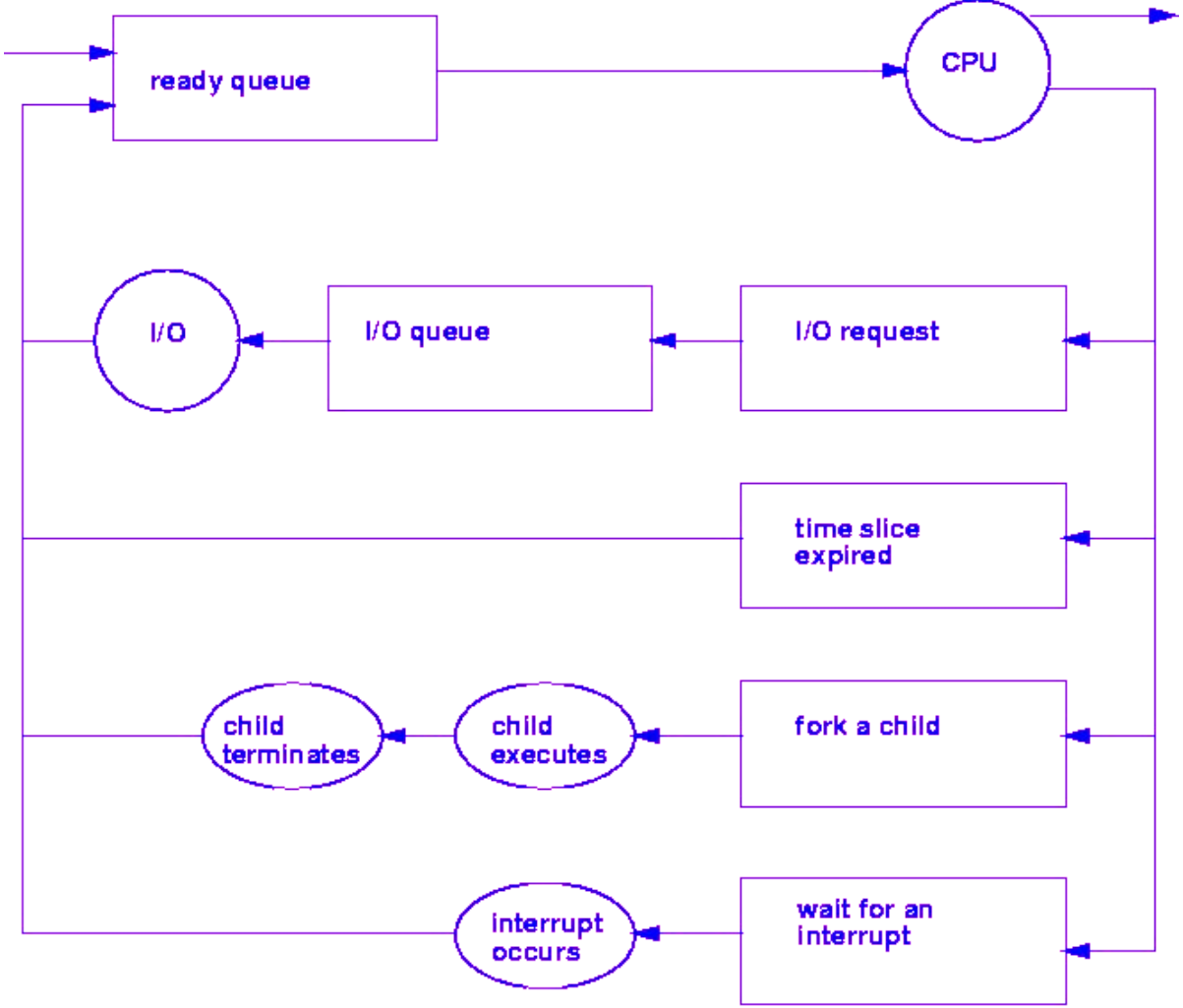


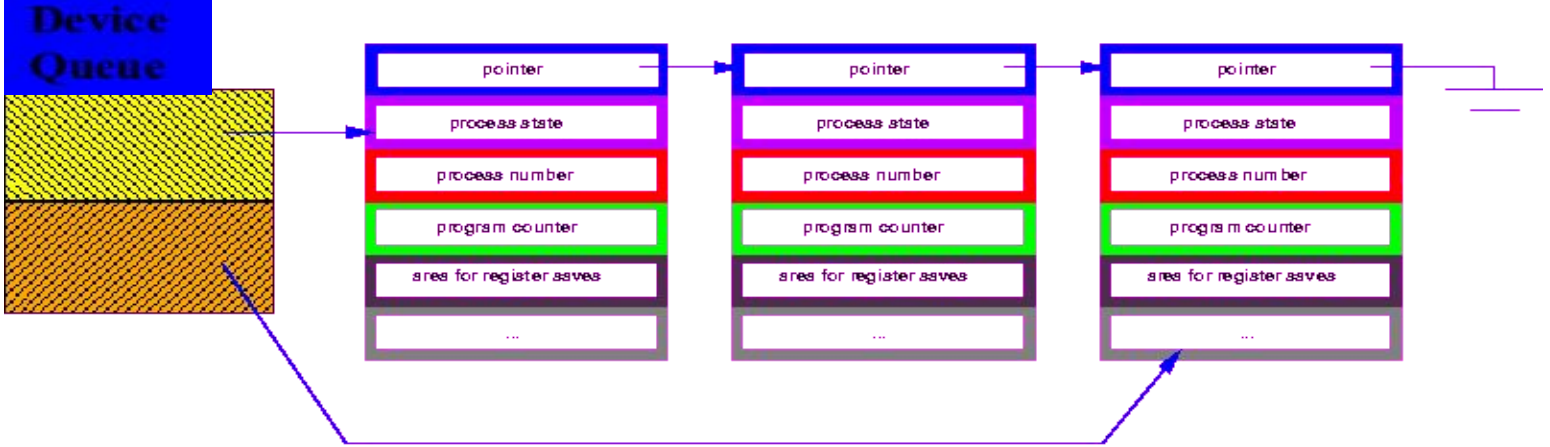
Process Scheduling Queues

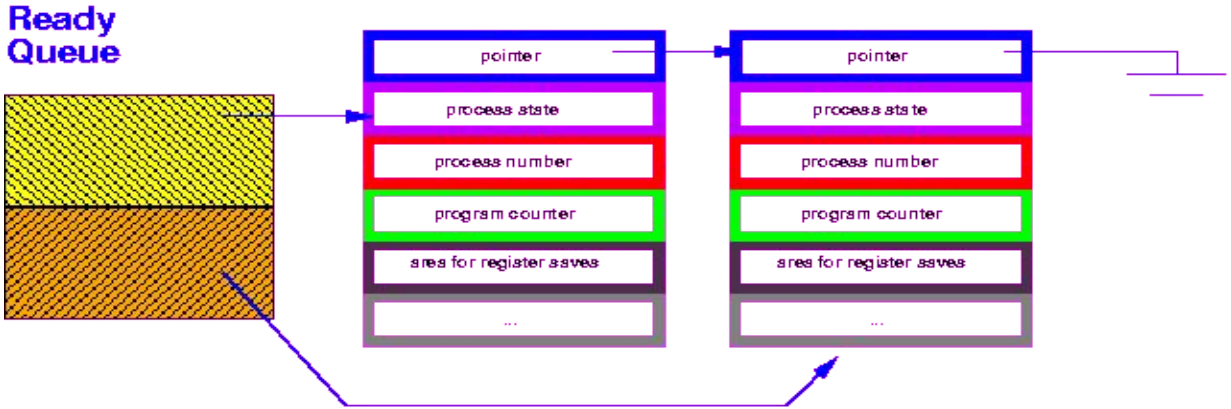
- Job Queue - set of all processes in the system
- Ready Queue - set of all processes residing in main memory, ready and waiting to execute.
- Device Queues - set of processes waiting for an I/O device.
- Process migration between the various queues.
- Queue Structures - typically linked list, circular list etc.

Representation of Process Scheduling

Process (PCB) moves from queue to queue




Device Queue
pointer
process state
process number
program counter
sres for register saves
...
pointer
process state
process number
program counter
sres for register saves
...
pointer
process state
process number
program counter
sres for register saves
...

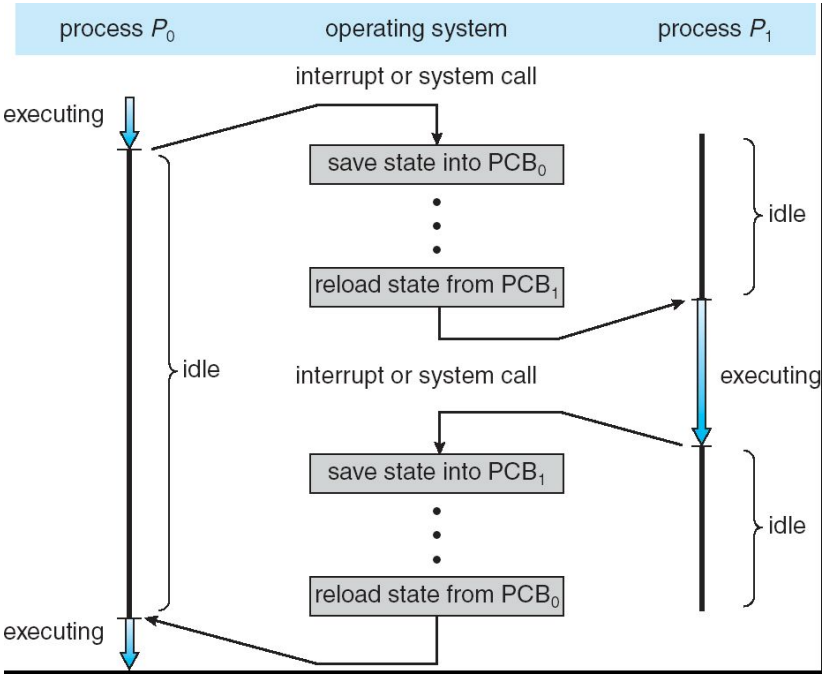

Ready Queue
pointer
process state
process number
program counter
sres for register saves
...
pointer
process state
process number
program counter
sres for register saves
...

Processes: Concurrency

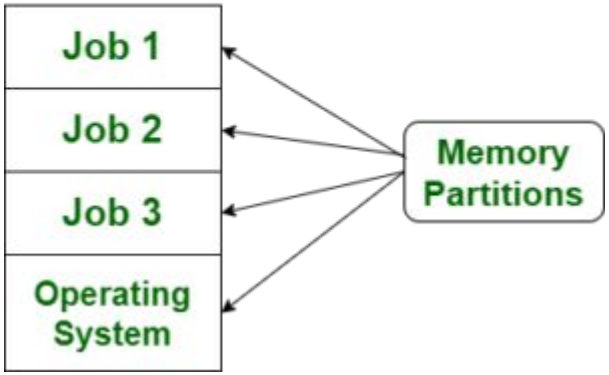
- Only one process (PCB) active at a time
  - Current state of process held in PCB:
    - “snapshot” of the execution and protection environment
  - Process needs CPU, resources
- Give out CPU time to different processes (Scheduling):
  - Only one process “running” at a time
  - Give more time to important processes
- Give pieces of resources to different processes (Protection):
  - Controlled access to non-CPU resources
  - E.g. Memory Mapping: Give each process their own address space

Processes: Context Switch

- Task that switches CPU from one process to another process
  - the CPU must save the PCB state of the old process and load the saved PCB state of the new process.
- Context-switch time is overhead
  - System does no useful work while switching
  - Overhead sets minimum practical switching time; can become a bottleneck
- Time for context switch is dependent on hardware support (1- 1000 microseconds).
- Code executed in kernel is overhead
  - Overhead sets minimum practical switching time

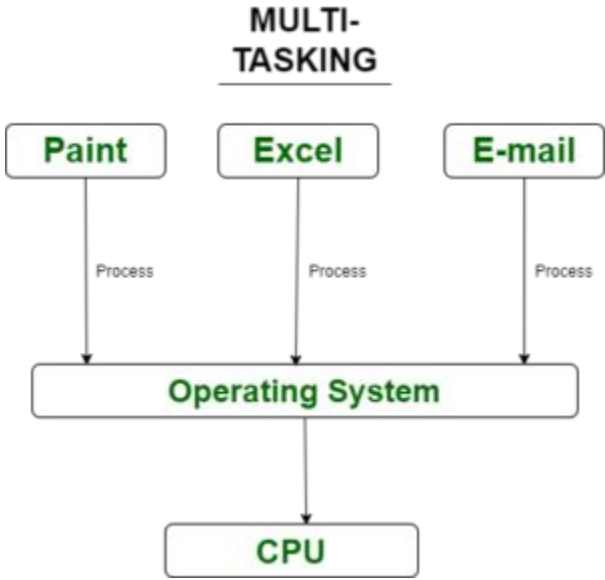


Difference between Multiprogramming and Multitasking



Multiporgramming

- CPU always has one to execute.
- The idea is to keep multiple jobs in main memory.
- If one job gets occupied with IO, CPU can be assigned to other job.



Multitasking

- Logical extension of multiprogramming. Multitasking is the ability of an OS to execute more than one **task** simultaneously on *a CPU machine.* These multiple tasks share common resources (like CPU and memory). In multi-tasking systems, the CPU executes multiple jobs by switching among them typically using a small time quantum

Difference between Multiprogramming and Multitasking

	Multiprogramming	Multitasking
1	Both of these concepts are for single CPU.	Both of these concepts are for single CPU.
2	Concept of Context Switching is used	Concept of Context Switching an time sharing is used
3	Operating system simply switches to, and executes, another job when current job needs to wait.	Switching happens when either allowed time expires or where there other reason for current process needs to wait
4	Increases CPU utilization by organising jobs .	Increases CPU utilization, it also increases responsiveness.
5	Reduce the CPU idle time for as long as possible.	Extend the CPU Utilization concept by increasing responsiveness Time Sharing.

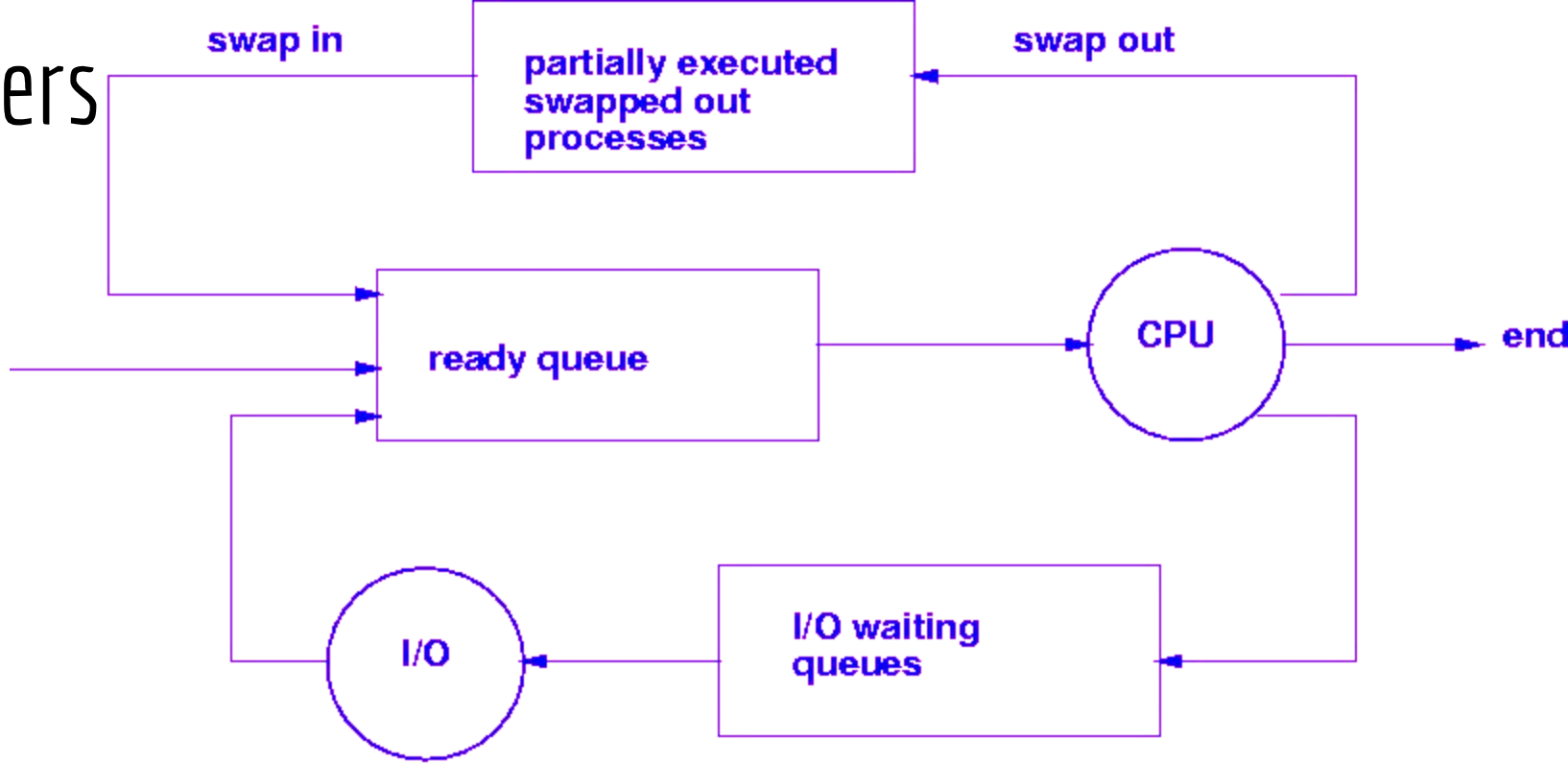
Preemptive and Cooperative Multitasking

- **Preemptive Multitasking - (Windows, Unix)**
  - Can initiate a context switching from the running process to another process
  - The operating system allows stopping the execution of the currently running process and allocating the CPU to some other process.
  - OS decides how long a process should execute before allowing another process to use the operating system.
  - A malicious program initiates an infinite loop, it only hurts itself without affecting other programs or threads.
  - Preemptive multitasking forces applications to share the CPU whether they want to or not.
- **Cooperative Multitasking - (Macos)**
  - Operating system never initiates context switching from the running process to another process.
  - A context switch occurs only when the processes voluntarily yield control periodically or when idle or logically blocked to allow multiple applications to execute simultaneously
  - All the processes cooperate for the scheduling scheme to work.
  - A malicious program can bring the entire system to a halt by busy waiting or running an infinite loop and not giving up control.
  - In cooperative multitasking, all programs must cooperate for it to work. If one program does not cooperate, it can hog the CPU.

Schedulers

- Process Scheduling handles the selection of a process for the processor on the basis of a scheduling algorithm and also the removal of a process from the processor.
- **Long Term Scheduler (or job scheduler)**
  - Selects processes from the storage pool in the secondary memory and loads them into the ready queue in the main memory for execution.
  - Controls the degree of multiprogramming.
  - It must select a careful mixture of I/O bound and CPU bound processes to yield optimum system throughput. If it selects too many CPU bound processes then the I/O devices are idle and if it selects too many I/O bound processes then the processor has nothing to do.
  - The job of the long-term scheduler is very important and directly affects the system for a long time.
- **Short Term Scheduler (or CPU scheduler)**
  - The short-term scheduler selects one of the processes from the ready queue using a scheduling algorithm and schedules them for execution.
  - The short-term scheduler executes much more frequently than the long-term scheduler as a process may execute only for a few milliseconds.
  - If it selects a process with a long burst time, then all the processes after that will have to wait for a long time in the ready queue.
  - **Dispatcher -** Gives control of the CPU to the process selected by the short-term scheduler.

Schedulers



- The medium-term scheduler swaps out a process from main memory.
- It can again swap in the process later from the point it stopped executing. This can also be called as suspending and resuming the process.
- Swapping is also useful to improve the mix of I/O bound and CPU bound processes in the memory